Cloud Computing Lab Report

Part A: Fundamentals & Setup | Part B: Web Application Deployment (AWS)

Part A: Fundamentals & Setup

Objective: Understand cloud basics and the AWS environment. **Outcome:** Understand AWS basics and EC2 deployment.

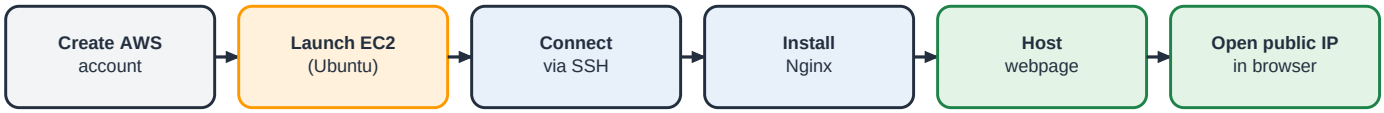

Figure 1: Workflow for Part A

Task 1. Create an AWS account

1. Open **aws.amazon.com** and click **Create an AWS Account**.
2. Enter email, account name and password; choose **Personal** account type.
3. Add contact details, a payment card (for identity verification) and verify the phone number with the OTP.
4. Select the **Basic (free) support plan**, then sign in to the **AWS Management Console**.
5. Set a region (top-right), for example **Asia Pacific (Mumbai) ap-south-1**.

PASTE SCREENSHOT HERE
AWS Console home / dashboard after login

Task 2. Learn basic cloud concepts (IaaS, PaaS, SaaS)

Model	Meaning	You manage	AWS example
IaaS (Infrastructure as a Service)	Virtual servers, storage and networking rented on demand.	OS, runtime, application, data	Amazon EC2, S3, VPC
PaaS (Platform as a Service)	Ready platform to run code; provider handles servers and OS.	Application and data only	Elastic Beanstalk, RDS
SaaS (Software as a Service)	Complete software used through a browser or app.	Only your usage and settings	Amazon WorkMail (also Gmail, Office 365)

Task 3. Launch a basic EC2 instance

1. Console search bar: **EC2** > **Instances** > **Launch instances**.
2. Name: **my-first-server**. AMI: **Ubuntu Server 22.04 LTS** (Free tier eligible).
3. Instance type: **t2.micro** (or **t3.micro** where t2 is unavailable).
4. Key pair: **Create new key pair**, name **mykey**, type RSA, format **.pem** (downloads automatically).
5. Network settings > **Create security group** with SSH (22) from **My IP** and HTTP (80) from **Anywhere**. Storage: 8 GiB gp3.
6. Click **Launch instance**. Wait until **Instance state = Running** and **Status check = 2/2 passed**.

PASTE SCREENSHOT HERE
EC2 Instances page showing Running + public IP

Task 4. Connect to EC2 using SSH

Why: SSH gives secure remote command-line access to the server. Run on your laptop (Linux/macOS terminal or Windows PowerShell):

```
chmod 400 mykey.pem
ssh -i mykey.pem ubuntu@<PUBLIC-IP>
```

Expected output (sample for reference)

```
Welcome to Ubuntu 22.04.x LTS (GNU/Linux 6.x aws x86_64)
ubuntu@ip-172-31-x-x:~$
```

PASTE SCREENSHOT HERE
Terminal showing ubuntu@ip-... prompt

Task 5. Install a simple web server (Nginx)

```
sudo apt update -y
sudo apt install nginx -y
sudo systemctl enable --now nginx
sudo systemctl status nginx
```

Expected output (sample for reference)

```
Active: active (running) since ...
Loaded: loaded (/lib/systemd/system/nginx.service; enabled)
```

PASTE SCREENSHOT HERE
Output of systemctl status nginx

Task 6. Host a simple webpage

Nginx serves files from `/var/www/html`. Replace the default page, then open `http://<PUBLIC-IP>` in a browser.

```
sudo tee /var/www/html/index.html > /dev/null <<'EOF'
<!DOCTYPE html>
<html><head><title>My First AWS Web Server</title>
<style>body{font-family:Arial;background:#232F3E;color:#fff;text-align:center;padding-top:80px}
h1{color:#FF9900}</style></head>
<body><h1>Hello from AWS EC2!</h1>
<p>Nginx web server running on Ubuntu.</p></body></html>
EOF
curl http://localhost
```

PASTE SCREENSHOT HERE
Browser showing 'Hello from AWS EC2!'

Part B: Web Application Deployment

Objective: Deploy a real application (WordPress + Amazon RDS MySQL) on the cloud

Outcome: Students learn cloud-based application hosting. The application tier (EC2) is separated from the database tier (RDS) and protected by Security Groups; an Elastic IP gives the site a fixed public address.

Figure 2: Workflow for Part B

System architecture flowchart

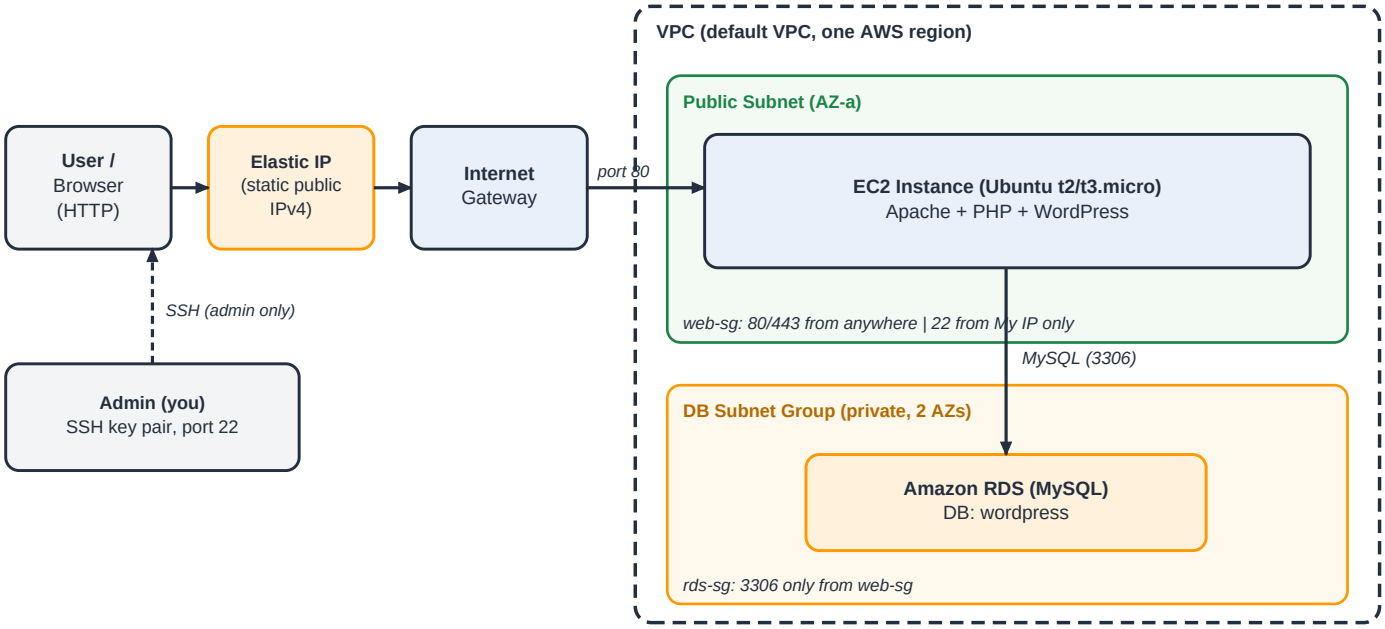

Figure 3: System architecture of the WordPress deployment (User, Elastic IP, Internet Gateway, EC2, RDS)

Task 1. Configure Security Groups (create first)

Console: **EC2 > Network & Security > Security Groups > Create security group**. Create **web-sg** first, then **rds-sg** and select web-sg as the source of the 3306 rule. Outbound: keep the default (all traffic).

Security Group	Type / Port	Source	Purpose
web-sg (EC2)	SSH 22	My IP only	Admin login, never open to the world
	HTTP 80, HTTPS 443	0.0.0.0/0	Public website traffic
rds-sg (RDS)	MySQL/Aurora 3306	web-sg (not an IP)	Only the web server can reach the database

PASTE SCREENSHOT HERE
web-sg inbound rules

PASTE SCREENSHOT HERE
rds-sg inbound rules

Task 2. Launch EC2 instance for the application

1. **EC2 > Launch instances**. Name: **wordpress-server**. AMI: **Ubuntu Server 22.04 LTS**; type **t2.micro / t3.micro**.
2. Key pair: reuse **mykey**. Network: default VPC, a **public subnet**, **Auto-assign public IP = Enable**. Security group: choose existing **web-sg**.
3. Launch and connect with SSH (as in Part A, Task 4).

PASTE SCREENSHOT HERE
EC2 instance wordpress-server running

Task 3. Configure Amazon RDS (MySQL)

1. Console: **RDS > Databases > Create database > Standard create > MySQL** (Engine 8.0.x).
2. Template: **Free tier** (if shown, otherwise Dev/Test with the smallest class). DB instance identifier: **wordpress-db**.
3. Master username: **admin**; set and note a strong password. Instance class: **db.t3.micro**; Storage: 20 GiB gp2/gp3.
4. Connectivity: default VPC, **Public access = No**, VPC security group = **rds-sg**. Additional configuration > **Initial database name = wordpress**.
5. Click **Create database**. When status is **Available**, open the database and copy the **Endpoint** (e.g. wordpress-db.xxxx.ap-south-1.rds.amazonaws.com).

PASTE SCREENSHOT HERE
RDS wordpress-db status Available

PASTE SCREENSHOT HERE
RDS endpoint & port 3306

Task 4. Install WordPress on EC2

Run on the EC2 instance (Apache web server + PHP + WordPress files):

```
sudo apt update -y
sudo apt install -y apache2 php libapache2-mod-php php-mysql php-curl php-gd php-mbstring php-xml php-zip mysql-client
cd /tmp && wget https://wordpress.org/latest.tar.gz && tar -xzf latest.tar.gz
sudo rm /var/www/html/index.html
sudo cp -r wordpress/* /var/www/html/
sudo chown -R www-data:www-data /var/www/html
sudo systemctl enable --now apache2
```

Expected output (sample for reference)

```
Active: active (running)  apache2.service
(Open http://<PUBLIC-IP>  -> WordPress language / setup page appears)
```

PASTE SCREENSHOT HERE
Apache active + WordPress setup page

Task 5. Connect WordPress with the RDS database

```
# 1) Test the database connection from EC2 (enter the RDS password)
mysql -h <RDS-ENDPOINT> -u admin -p -e "SHOW DATABASES;"
# 2) Create the config file and edit it
cd /var/www/html && sudo cp wp-config-sample.php wp-config.php
sudo nano wp-config.php
   define('DB_NAME',     'wordpress');
   define('DB_USER',     'admin');
   define('DB_PASSWORD', '<YOUR-RDS-PASSWORD>');
   define('DB_HOST',     '<RDS-ENDPOINT>');
# 3) Restart Apache
sudo systemctl restart apache2
```

Expected output (sample for reference)

```
+--------------------+
| Database           |
+--------------------+
| information_schema |
| wordpress          |
+--------------------+
```

Now open `http://<PUBLIC-IP>`, choose the language, enter Site Title, admin username/password/email, and click **Install WordPress**. A "Success!" page confirms WordPress is saving data in RDS.

PASTE SCREENSHOT HERE
mysql SHOW DATABASES output

PASTE SCREENSHOT HERE
WordPress 'Success!' page

Task 6. Assign an Elastic IP

1. Console: **EC2 > Network & Security > Elastic IPs > Allocate Elastic IP address > Allocate**.
2. Select the new address > **Actions > Associate Elastic IP address** > Resource type **Instance** > choose **wordpress-server** > **Associate**.
3. Open `http://<ELASTIC-IP>`. The address stays the same even after the instance is stopped and started. (Also set the WordPress Address/Site URL to this IP in **Settings > General**.)
4. Cost note: public IPv4 addresses are charged hourly; release the Elastic IP when you finish.

PASTE SCREENSHOT HERE
Elastic IP associated with instance

PASTE SCREENSHOT HERE
Final WordPress site via Elastic IP

Clean-up (to avoid charges)

After capturing screenshots: **terminate** both EC2 instances, **delete** the RDS database (skip final snapshot), **release** the Elastic IP, and delete unused Security Groups.